

Figure 16: Regression task: RMSE and satisfaction level over time.

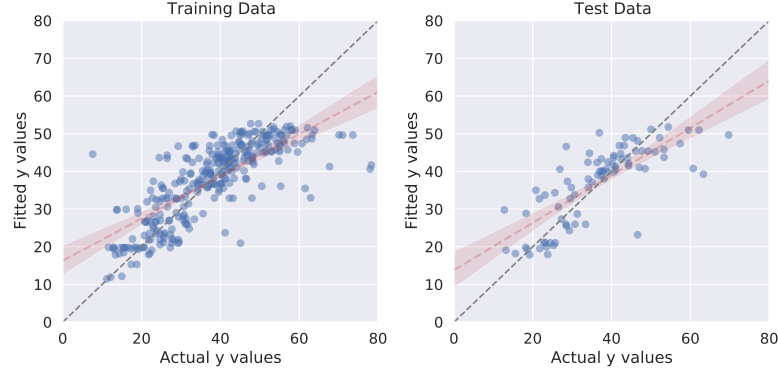


Figure 17: Visualization of LTN solving a regression problem.

scales how strict the smooth equality is.<sup>26</sup> In our experiments, we use  $\alpha = 0.05$ .

$\mathcal{G}(f^*(x) \mid \theta) = \text{MLP}_\theta(x)$ , where  $\text{MLP}_\theta$  is a multilayer perceptron which ends in one neuron corresponding to a price prediction, with a linear output layer (no activation function).

#### Learning:

The theory is constrained by the parameters of the model of  $f^*$ . LTN is used to estimate such parameters by maximizing the satisfaction of the knowledge-base, in the usual way. Approximating  $\forall$  using  $A_{pME}$  with  $p = 2$ , as before, we randomly split the data set into 330 examples for training and 84 examples for testing. Figure 16 shows the satisfaction level over 500 epochs. We also plot the Root Mean Squared Error (RMSE) between the predicted prices and the labels (i.e. actual prices, also known as target values). We visualize in Figure 17 the strong correlation between actual and predicted prices at the end of one of the runs.

#### 4.6. Unsupervised Learning (Clustering)

In unsupervised learning, labels are either not available or are not used for learning. Clustering is a form of unsupervised learning whereby, without labels, the data is characterized by constraints

<sup>26</sup>Intuitively, the smooth equality is  $\exp(-\alpha d(\mathbf{u}, \mathbf{v}))$ , where  $d(\mathbf{u}, \mathbf{v})$  is the Euclidean distance between  $\mathbf{u}$  and  $\mathbf{v}$ . It produces a 1 if the distance is zero; as the distance increases, the result decreases exponentially towards 0. In case an exponential decrease is undesirable, one can adopt the following alternative equation:  $\text{eq}(\mathbf{u}, \mathbf{v}) = \frac{1}{1 + \alpha d(\mathbf{u}, \mathbf{v})}$ .

alone. LTN can formulate such constraints, such as:

- clusters should be disjoint,
- every example should be assigned to a cluster,
- a cluster should not be empty,
- if the points are near, they should belong to the same cluster,
- if the points are far, they should belong to different clusters, etc.

**Domains:**

points, denoting the data to cluster.  
points\_pairs, denoting pairs of examples.  
clusters, denoting the cluster.

**Variables:**

$x, y$  for all points.  
 $\mathbf{D}(x) = \mathbf{D}(y) = \text{points}$ .  
 $\mathbf{D}(c) = \text{clusters}$ .

**Predicates:**

$C(x, c)$ , the truth degree of a given point belonging in a given cluster.  
 $\mathbf{D}_{\text{in}}(C) = \text{points, clusters}$ .

**Axioms:**

$$\forall x \exists c C(x, c) \quad (42)$$

$$\forall c \exists x C(x, c) \quad (43)$$

$$\forall (c, x, y : |x - y| < \text{th}_{\text{close}}) (C(x, c) \leftrightarrow C(y, c)) \quad (44)$$

$$\forall (c, x, y : |x - y| > \text{th}_{\text{distant}}) \neg(C(x, c) \wedge C(y, c)) \quad (45)$$

Notice the use of guarded quantifiers: all the pairs of points with Euclidean distance lower (resp. higher) than a value  $\text{th}_{\text{close}}$  (resp.  $\text{th}_{\text{distant}}$ ) should belong in the same cluster (resp. should not).  $\text{th}_{\text{close}}$  and  $\text{th}_{\text{distant}}$  are arbitrary threshold values that define some of the closest and most distant pairs of points. In our example, they are set to, respectively, 0.2 and 1.0.

As done in the example of Section 4.2, the clustering predicate has mutually exclusive satisfiability scores for each cluster using a `softmax` layer. Therefore, there is no explicit constraint about clusters being disjoint.

**Grounding:**

$$\mathcal{G}(\text{points}) = [-1, 1]^2.$$

$$\mathcal{G}(\text{clusters}) = \mathbb{N}^4, \text{ we use one-hot vectors to represent a choice of 4 clusters.}$$

$$\mathcal{G}(x) \in [-1, 1]^{m \times 2}, \text{ that is, } x \text{ is a sequence of } m \text{ points. } \mathcal{G}(y) = \mathcal{G}(x).$$

$$\text{th}_{\text{close}} = 0.2, \text{th}_{\text{distant}} = 1.0.$$

$$\mathcal{G}(c) = \langle [1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1] \rangle.$$

$$\mathcal{G}(C \mid \theta) : x, c \mapsto c^\top \cdot \text{softmax}(\text{MLP}_\theta(x)), \text{ where MLP has 4 output neurons corresponding to the 4 clusters.}$$

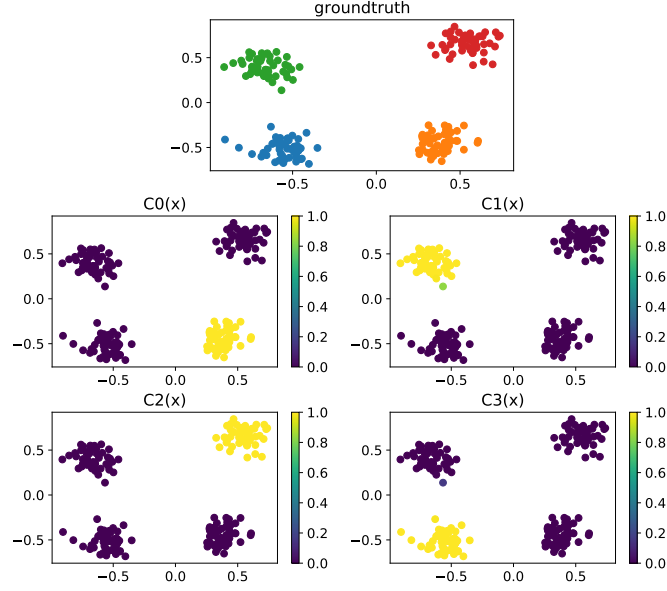


Figure 18: LTN solving a clustering problem by constraint optimization: ground-truth (top) and querying of each cluster  $C0$ ,  $C1$ ,  $C2$  and  $C3$ , in turn.

#### Learning:

We use the stable real product configuration to approximate the logical operators. For  $\forall$ , we use  $A_{pME}$  with  $p = 4$ . For  $\exists$ , we use  $A_{pM}$  with  $p = 1$  during the first 100 epochs, and  $p = 6$  thereafter, as a simplified version of the schedule used in Section 4.4. The formula aggregator is approximated by  $A_{pME}$  with  $p = 2$ . The model is trained for a total of 1000 epochs using the Adam optimizer, which is sufficient for LTN to solve the clustering problem shown in Figure 18. Ground-truth data for this task was generated artificially by creating 4 centers, and generating 50 random samples from a multivariate Gaussian distribution around each center. The trained LTN achieves a satisfaction level of the clustering constraints of 0.857.

#### 4.7. Learning Embeddings with LTN

A classic example of Statistical Relational Learning is the smokers-friends-cancer example introduced in [55]. Below, we show how this example can be formalized in LTN using semi-supervised embedding learning.

There are 14 people divided into two groups  $\{a, b, \dots, h\}$  and  $\{i, j, \dots, n\}$ . Within each group, there is complete knowledge about smoking habits. In the first group, there is complete knowledge about who has and who does not have cancer. Knowledge about the friendship relation is complete within each group only if symmetry is assumed, that is,  $\forall x, y (friends(x, y) \rightarrow friends(y, x))$ .